TAKE-HOME TEST

Room Booking App

18 October 2026

STACK	EXPECTED TIME	DEADLINE	AI TOOLS
Django · PostgreSQL · React	3 to 4 hours	3 days after receipt	Allowed

Build a small meeting-room booking app with Django, PostgreSQL and React. This brief says what the app must do, not how to build it: the design decisions are yours.

The task

Our office in Doha has a few meeting rooms. People keep double-booking them. Build a small web app where a logged-in user can:

- see the rooms and the bookings for a room on a given day
- book a room for a time slot
- cancel their own bookings

The backend is a Django REST API on PostgreSQL. The frontend is a React app that uses that API. Keep it small, but make it correct: we care more about the rules below working under real conditions than about features or looks.

Requirements

1. Users log in with a username and password. No sign-up page is needed: create a few demo users and rooms.
2. Each room has a name and a capacity.
3. A booking has a room, a title, a start time, an end time and the person who made it.
4. A booking lasts at least 15 minutes and at most 4 hours. It cannot start in the past.
5. A room can never be double-booked. A meeting ending at 10:00 and another starting at 10:00 do not clash.
6. Rule 5 must still hold when two people try to book the same slot at the same moment.
7. Only the person who made a booking can cancel it. A cancelled slot can be booked again.
8. The office runs on Doha time. Users see and enter times in Doha time, whatever their computer is set to.
9. When a booking is refused, the user sees a clear reason, for example that the room is already taken at that time.
10. Viewing a day with many bookings should stay fast.

Engineering standards

We will review these areas closely. How you meet them is your choice: explain your main decisions in the README.

Area	What we expect
Database design	A normalized schema: each fact stored once, clear relations, and database constraints so invalid data cannot be saved, even by code that skips your validation.
Indexing	Indexes that match the queries the app really runs. Say in the README which ones you added and why.
Authentication	Every endpoint except login needs a logged-in user. Pick an auth method that suits a React app and explain the choice. Passwords are never stored or logged in plain text.
Security	The server enforces every rule; the frontend is never trusted. Users can only change their own data. Inputs are validated. No secrets in the repo, safe production settings, and protection against common attacks (SQL injection, XSS, CSRF, open CORS).
API organisation	A consistent REST API: clear resource URLs, correct HTTP methods and status codes, one error format, pagination for lists. List your endpoints in the README, or provide OpenAPI docs.
Performance	The API and the UI stay fast as data grows: no query per row, filtering done in the database, and no loading everything into the browser. Tell us how you checked.

Stack and quality

- Backend: Django (Django REST Framework is fine). Database: PostgreSQL, not SQLite. Frontend: React.
- Someone else must be able to run the whole thing from your README in a few commands. Docker Compose is welcome.
- Write automated tests for the rules you think matter most, and tell us why you chose them.
- Commit as you go, so we can follow how the work progressed. One big final commit tells us nothing.
- No secrets or passwords committed, apart from clearly marked local demo values.
- Out of scope: sign-up, password reset, emails, recurring bookings, deployment, polished design. Clean and usable is enough.

Using AI

Use any AI tool you like (ChatGPT, Claude, Copilot, Cursor and so on). We use them too. What we want to see is how you work with it: what you accept, what you check, and what you fix.

Add a short `AI_NOTES.md` to the repo:

- which tools you used, and for which parts
- at least two places where the AI was wrong or weak, and what you did about it
- one decision you made yourself, against or without the AI's suggestion, and why

You own every line you submit. In the follow-up call we will ask you to explain any part of the code.

How to submit and what happens next

Send us a link to a Git repository (GitHub or GitLab, public or shared with us). Its README should cover:

- how to run the app and the tests, and the demo logins
- your main design decisions, in a few lines each
- what you would improve or do differently with more time

After we review it, we book a 45-minute video call. You walk us through your code, answer questions about it, and make one small change live while sharing your screen. AI is allowed in that call too.

> If something in this brief is unclear, make a reasonable choice and write it down in the README. That is part of the test.

Good luck. We look forward to seeing how you approach it.